

Reviewer Pack — Minimal Quadratic Intersections and Communication Rank Growth...

Entry 7ea8f6976de3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 02:58:02 UTC

Minimal Quadratic Intersections and Communication Rank Growth

Entry ID: 7ea8f6976de3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 02:58:02 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (specifically, Intersection Theory) **Field B** (complexity object): Communication Complexity

Statement:

The minimal number of quadratic intersections in the moduli space of genus-zero curves associated with a communication protocol P is $\Theta(\log(n) \cdot \kappa(P))$, where $\kappa(P)$ is the communication rank of P .

Rationale (proposer's reasoning):

Intersection theory provides a rich framework for studying geometric properties of varieties, and by associating these properties with communication protocols, we may uncover non-trivial relationships that could be used to prove complexity lower bounds. The quadratic intersection number can serve as a potential structural invariant that captures the essence of the complexity of communication tasks.

Taxonomy category: IntersectionTheory × CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): be2367632576d5ca

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all protocols P with communication rank $\kappa(P)$ up to $n=40$, the ratio of minimal quadratic intersections (MI) to $\log(n)$ is within 10% of $\kappa(P)$, and no seed results in $MI > 1.2 \cdot \kappa(P)$. It is falsified if any protocol's MI ratio deviates by more than 20% from $\kappa(P)$ or if any seed produces an $MI > 1.2 \cdot \kappa(P)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal quadratic intersections" AND "communication rank" AND algebraic geometry" - "genus-zero curves" AND "communication complexity" AND intersection theory" - " $\Theta(\log(n) \cdot \kappa(P))$ " AND algebraic geometry AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_protocol(n):
        # Example protocol: a simple binary protocol with rank n
        return [random.randint(0, 1) for _ in range(n)]

    def communication_rank(protocol):
        # Example ranking: number of 1s in the protocol
        return sum(protocol)

    def quadratic_intersections(rank):
        # Example computation: quadratic intersections are proportional to rank^2
        return rank ** 2

    n_max = 40
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in range(5, n_max + 1):
        protocol = communication_protocol(n)
        rank = communication_rank(protocol)
        mi = quadratic_intersections(rank)

        if mi > 1.2 * rank:
            conjecture_holds = False
            counterexample = f"n={n}, rank={rank}, MI={mi}"
            break
```

```

        instances_tested += 1
        total_metric_value += mi / math.log(n)

mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0.0

return {
    "metric_name": "minimal_quadratic_intersections",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if abs((r["metric_value"] - math.log(r["n_max"])) / math.log(r["n_max"] - 1)) > 0.2) > 0:
        print(f"RESULT: FALSIFIED counterexample=\"MI ratio deviates by more than 20% from  $\kappa(P)$ \")")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

reexample': 'n=5, rank=3, MI=9'}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.6213349345596119, 'instances_tested': 0}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}
TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}

```

```

TRIAL: {'metric_name': 'minimal_quadratic_intersections', 'metric_value': 0.0, 'instances_tested': 0}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_284b4c81.py", line 85, in <module>
    elif sum(1 for r in results if abs((r["metric_value"] - math.log(r["n_max"])) / math.log(r["n_max"]
r["communication_rank"]) > 0.2 * r["communication_rank"]) >= len(results) * 0.8:
    ~~~~~
File

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data that could confirm or falsify the conjecture.
 | next: Review and debug the test code to ensure it can complete without crashing, then
 rerun the test with a protocol up to n=40.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13405
2	preregistration	ollama_remote	glm4:latest	0	0	11379
3	novelty	ollama_remote	glm4:latest	0	0	8470
4	novelty	ollama_remote	glm4:latest	0	0	11775
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16815
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27166
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15731
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10238
9	judge	ollama_remote	glm4:latest	0	0	8915

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123894 ms total latency. Provider mix:
 {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7ea8f6976de3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7ea8f6976de3.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7ea8f6976de3.tar.gz` (if generated)